Department of Engineering & Technology
Gurugram University, Gurugram

Synopsis
on
Sentinel RAG

Akshu Grewal (231030050004)
Ishantnu (231030050023)
Anish Singh Rawat (231030050007)
Bachelor of Technology
Computer Science & Engineering (Specialization: Artificial Intelligence)

Table of Contents

- Introduction
- Problem Statement
- Project Objectives
- Proposed Methodology
- Tools and Technologies
- Expected Outcome
- Future Scope
- References

Introduction

SentinelRAG is a Retrieval-Augmented Generation (RAG) system that focuses on **engineering reliability** rather than only chat quality. It It tackles the “black-box” issue in AI deployments by making every step observable, measurable, and testable.

Why RAG?

- Answers grounded in documents (less hallucination).
- Easy to update knowledge without retraining.
- Works offline with local models (Ollama).

Core idea: Retrieve → Validate → Generate → Measure.

What makes it “Sentinel”?

- **Router + Grader** for agentic decisions.
- **Web fallback** when local docs fail.
- **CI gate** to prevent silent regressions.

Goal:production-like guardrails in a student project.

What we measure

- **TTFT** (Time-To-First-Token) for UX latency.
- Token counts & **simulated cost** for scaling.
- Faithfulness score using **RAGAS**.

Output:traces + metrics + pass/fail gate.

Problem Statement

In many RAG/chatbot setups, engineers can't tell **where** things broke. The result is poor debugging, unpredictable latency, and silent quality regressions.

1 Pinpoint failures
Hard to separate **retrieval errors** from **LLM hallucinations** .

2 Measure real UX
Without metrics like **TTFT** , latency bottlenecks remain invisible.

3 Prevent regressions
Prompt/data updates can degrade accuracy with no automated warning.

4 Handle information gaps
Outdated local docs lead to wrong answers unless there's a fallback strategy.

SentinelRAG addresses these with 3 "guardrails"

Guardrail A — Observability
Phoenix traces each node execution (retrieval / grading / generation) to remove guesswork.

Guardrail B — Self-Correction
A Grader node blocks irrelevant context and routes to web fallback instead of hallucinating.

Guardrail C — QA Gate
RAGAS faithfulness gate stops deployment when score **< 0.85**

Project Objectives

1) Autonomous Self-Correction

- LLM-based Grader filters irrelevant document chunks.
- Prevents hallucinated answers from bad context.
- Routes to fallback when retrieval quality is low.

Deliverable: conditional routing in LangGraph (grade → generate / web_search).

2) Web Intelligence (Zero-Cost)

- DuckDuckGo search for fresh URLs.
- Scrapling **StealthyFetcher** for stealth scraping.
- Unified local + web context before generation.

Deliverable: web fallback node that appends scraped text into context.

3) Observability + QA Gate

- OpenTelemetry export to **Arize Phoenix** dashboard.
- Track TTFT + token cost estimate.
- **RAGAS** faithfulness eval gate with threshold **0.85.**

Deliverable: eval_gate.py returns non-zero exit code to block CI/CD.

Proposed Methodology

1

Ingest + Embed

Chunk docs → embed locally with **Ollama (llama3.2)** → store in **ChromaDB.**

2

Retrieve

Similarity search **(k=3)** returns candidate context chunks for the query.

3

Grade Relevance

LLM grader outputs **yes/no.** If "no" → route to web fallback (self-correction).

4

Web Fallback

DuckDuckGo search → Scrapling **StealthyFetcher** → append clean text into context.

5

Generate + Collect

Generate answer + compute **TTFT**, token counts, simulated cost; traces exported to Phoenix.

Tools and Technologies

Everything runs locally (privacy-first) with open-source tooling.

Python 3.10+ | Ollama · llama3.2 | LangGraph | ChromaDB | DuckDuckGo-Search | Scrapling (StealthyFetcher) | OpenTelemetry OTLP | OpenInference | RAGAS | Arize Phoenix

Core Runtime

- **ChatOllama** for generation (temperature=0).
- **OllamaEmbeddings** for local embeddings.
- Chroma persistent DB (./chroma_db).

```
pip install -r requirements.txt
ollama pull llama3.2
```

Observability

- Phoenix dashboard: **http://localhost:6006**
- OTLP exporter: /v1/traces
- Node-level trace visibility for debug.

```
python launch_phoenix.py
python rag_system.py
```

Evaluation Gate

- Golden dataset: golden_dataset.json
- Metric: **faithfulness**
- Threshold: **0.85** (fail build below this).

```
python eval_gate.py
exit code1 if score < 0.85
```

Expected Outcome

- **Self-healing** responses via relevance grading + web fallback.
- **Operational transparency** in Phoenix: execution path + per-node timings.
- **Deterministic QA:** CI gate blocks changes when faithfulness < 0.85.
- **Cost & privacy blueprint:** fully local, zero recurring API cost.
- **Benchmark insights:** TTFT + throughput on local hardware.

Future Scope

- Replace seeded text with real corpus ingestion (PDFs, notes, policies).
- Improve grader: multi-doc scoring + chunk filtering (not just yes/no).
- Add p50/p95 latency dashboards + alerts.
- Stronger eval suite: additional RAGAS metrics (context recall, answer relevancy).
- Containerize + integrate into GitHub Actions as a real CI step.

.

References

- LangChain-AI. (2024). LangGraph: Building Stateful, Multi-Agent Applications. [Online]. Available: https://python.langchain.com/docs/langgraph
- Arize AI. (2024). Phoenix: Open-Source AI Observability and Evaluation. [Online]. Available: https://phoenix.arize.com/
- Espejel, J., et al. (2023). "RAGAS: Automated Evaluation of Retrieval Augmented Generation." arXiv preprint arXiv:2309.15217.
- Ollama Team. (2024). Ollama: Get up and running with large language models locally. [Online]. Available: https://ollama.com/
- Scrapling Documentation. (2026). Stealthy Web Extraction and Anti-Bot Bypass for AI Agents. [Online]. Available: https://github.com/D4Vinci/Scrapling
- SentinelRAG Technical Documentation. (2026). RAG-Ops-Sentinel: Production-Grade Observability Pipeline. GitHub Repository.